9. Proposed Solution

Date	28 June 2026
Team ID	SWTID-2026-9775
Project Name	EduGenie – Google Gemini Powered Learning Assistant
Maximum Marks	5 Marks

Project Overview

Objective	**Give students a single AI-powered assistant that answers questions, explains concepts at the right depth, generates practice quizzes, summarizes notes, and builds a personalized learning roadmap — all backed by Google Gemini with an offline/local-model fallback.**
Scope	A FastAPI web application with five request/response modules (chat, explanation, quiz, summary, learning path), a Jinja2/HTML/CSS/JS frontend, JSON-file persistence for query/response history, and a Dockerized deployment path.

Problem Statement

Description	**Students lack a single, adaptive tool that can explain a concept at their level, quiz them on it, summarize their notes, and tell them what to study next — they currently stitch this together from multiple generic tools.**
Impact	Solving this reduces study friction and time-to-understanding, gives students a low-cost self-testing mechanism, and helps them plan their learning path with less guesswork.

Proposed Solution

Approach	**Use Google Gemini (with dynamic best-model resolution and retry/backoff) as the primary generative engine, prompted with strict JSON response schemas per feature, and a local HuggingFace LaMini-Flan-T5 model plus a built-in mock/demo mode as fallbacks so the app is always usable.**
Key Features	• AI Q&A chat with structured answer/explanation/example/key-points • Concept explanation at simple / medium / advanced levels • MCQ quiz generation (1–15 questions) with per-question explanations • Text/notes summarization with bullet points and keywords • Personalized learning roadmap with resources and a step-by-step plan • JSON-file logging of every query and AI response for traceability

Resource Requirements

Resource Type	Description	Specification / Allocation
Computing Resources	CPU / GPU for running the app and the local T5 model	CPU-only by default (utils/local_t5_service.py auto-detects CUDA and uses GPU only if available)
Memory	RAM to hold the local LaMini-Flan-T5-248M/783M model in memory when loaded	≈ 2–4 GB recommended
Storage	Disk space for model weight cache, Docker image and JSON data files	A few GB (mostly HuggingFace model cache)
Frameworks	Python web frameworks used	FastAPI, Uvicorn, Jinja2
Libraries	Core dependencies (requirements.txt)	google-generativeai, transformers, torch, pydantic, python-dotenv, httpx, python-multipart, pytest
Development Environment	IDE / tooling	Any Python IDE (VS Code / PyCharm), Docker Desktop, Git
Data	Source of content	No external dataset — all content is generated live via Gemini / local T5; JSON store is seeded empty at runtime by init_db()